EST-744

Embeddings y Aprendizaje de Representaciones

Juan Zamora O.

Deep Learning





PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

El Problema: El Texto es Simbólico

¿Cómo alimentamos palabras a redes neuronales?

El desafío fundamental es que las palabras son símbolos discretos sin estructura numérica inherente. Las redes neuronales operan sobre números y geometrías continuas. Necesitamos construir un puente.

Enfoques tradicionales (insuficientes):

- **Codificación one-hot**: “gato” → [0, 0, ..., 1, 0, ..., 0]
  - Vocabulario de 10,000 palabras = 10,000 dimensiones
  - No captura relaciones semánticas (todas las palabras equidistantes)
  - Vectores enormes y dispersos (99.99% ceros)
  - Desperdicia la estructura subyacente del lenguaje

Insight fundamental: Aprender representaciones densas donde el significado se codifica en la **geometría** del espacio de vectores, no en la presencia/ausencia de dimensiones

La Idea Central

Un **embedding** es un mapeo aprendido que transforma cada palabra en un vector denso de dimensionalidad mucho menor (típicamente 50-300).

```
"gato"   → [0.23, -0.15, 0.08, ..., 0.42]    (300 dimensiones)
"perro"  → [0.25, -0.13, 0.07, ..., 0.41]
"rey"    → [0.12,  0.45, -0.22, ..., 0.61]
"reina"  → [0.10,  0.50, -0.20, ..., 0.58]
```

Propiedad emergente: Palabras similares tienen vectores cercanos en el espacio

¿Por qué funciona? La distancia euclidiana entre vectores codifica similitud semántica:

$$\text{similaridad} \propto \frac{1}{1 + \text{distancia}}$$

La Hipótesis Distribucional

“Conocerás una palabra por la compañía que frecuenta” (Firth, 1957)

Principio linguístico: Palabras que aparecen en contextos similares tienen significados similares

Ejemplo práctico:

- “gato” aparece con: “mascota”, “ronroneo”, “felino”, “suave”
- “perro” aparece con: “mascota”, “ladrido”, “canino”, “leal”
- **Contexto compartido** → los vectores deberían ser cercanos

Este principio es el fundamento teórico de todos los métodos modernos de embeddings.

Idea Central del Modelo Skip-Gram

En lugar de tratar de predecir la siguiente palabra (como en un modelo de lenguaje), **predecimos las palabras contextuales a partir de la palabra objetivo**.

```
Oración: "Adoro mi [gato] muy tierno"
         Contexto: "Adoro", "mi", "muy", "tierno"
         Objetivo: "gato"

Tarea: Dado "gato" → predecir palabras contextuales
```

Arquitectura de la red

1. Entrada: palabra objetivo codificada one-hot
2. Capa oculta: matriz de embeddings $W \in \mathbb{R}^{|V| \times d}$ (|V| = vocab, d = dim embedding)
3. Salida: distribución de probabilidad sobre palabras contextuales

Truco elegante: Después del entrenamiento, descartamos la capa de salida y usamos los **pesos de la capa oculta como embeddings**.

Proceso de Entrenamiento

1. **Extracción de pares**: Recorrer el corpus con ventana de contexto (ej: 5 palabras a cada lado)
2. **Construcción de ejemplos**: Para cada palabra objetivo, generar pares (objetivo, contexto)
3. **Optimización**: Maximizar probabilidad de palabras contextuales dada la palabra objetivo

$$\mathcal{L} = -\sum_{t=1}^{T} \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j}|w_t)$$

4. **Emergencia de estructura**: La red aprende automáticamente que palabras similares tienen vectores similares
5. **Resultado**: Vectores que capturan relaciones semánticas y sintácticas

Operaciones Vectoriales Significativas

Los embeddings capturan no solo similitud, sino **relaciones funcionales**:

$$\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$$

¿Por qué funciona?

Piensa en cada dimensión del embedding como codificar un atributo semántico (no interpretable directamente, pero existe):

- “rey” tiene componentes de “autoridad” + “masculinidad”
- “hombre” codifica principalmente “masculinidad”
- Restar “hombre” elimina el componente de masculinidad
- Sumar “mujer” añade “feminidad”
- Resultado: “autoridad” + “feminidad” ≈ “reina”

Advertencia importante: No todas las relaciones se preservan perfectamente. Las operaciones vectoriales son aproximadas, especialmente para conceptos abstractos.

Comparación de Alternativas

No se entrena embeddings desde cero; se usan modelos pre-entrenados en corpus masivos:

Método	Dimensiones	Corpus	Ventajas	Limitaciones
Word2Vec	300	Google News (3B palabras)	Rápido, intuitivo, eficiente	Palabras OOV desconocidas
GloVe	300	Wikipedia + Gigaword (6B)	Equilibra contexto local y global	Más lento en entrenamiento
FastText	300	Common Crawl (840B)	Maneja OOV via subpalabras	Archivos grandes (~7 GB)

¿Cuál elegir?

- **Word2Vec**: Baseline rápida, buena para tareas generales
- **GloVe**: Si importa equilibrio entre contexto inmediato y correlaciones globales
- **FastText**: Cuando hay palabras raras, idiomas morfológicamente ricos, o necesitas robusted

Para esta sesión: Usaremos `gensim` (Word2Vec) por simplicidad e interpretabilidad

Integrando Embeddings en Modelos Deep Learning

En lugar de mandar one-hot encoding a la red, usamos una **capa de embeddings** que actúa como tabla de búsqueda:

```
keras.layers.Embedding(
    input_dim=vocab_size,      # ¿Cuántas palabras en vocabulario?
    output_dim=embedding_dim,   # Dimensionalidad del embedding (ej: 128)
    input_length=max_length     # Largo máximo de secuencia
)
```

Flujo de datos:

1. **Tokenización**: “el gato duerme” → [3, 42, 157]
2. **Embedding layer**: [3, 42, 157] → matriz de shape (3, embedding_dim)
   - Fila 0: embedding de palabra con índice 3
   - Fila 1: embedding de palabra con índice 42
   - Fila 2: embedding de palabra con índice 157
3. **Propagación**: Esta matriz se pasa a capas posteriores (LSTM, CNN, etc.)

Dos estrategias para obtener Embeddings:

1. **Aprender from scratch**: Si tienes mucho datos etiquetados
   - La capa de embeddings se optimiza junto con toda la red
   - `trainable=True`
2. **Transfer learning** (recomendado): Cargar pre-entrenados y fine-tunear
   - Inicializar con Word2Vec/GloVe
   - `trainable=True` solo si dataset es suficientemente grande
   - Típicamente: `trainable=False` para datasets pequeños

Del Nivel de Palabra al Documento

Los embeddings word-level son poderosos, pero ¿cómo representamos documentos completos?

Opción 1: Promediado Simple

```
oración = "el gato es feliz"
embedding_documento = mean([emb(el), emb(gato), emb(es), emb(feliz)])
```

Pros: Simple, rápido, funciona sorprendentemente bien

Contras: Pierde orden de palabras (“no bueno” = “bueno no”)

Opción 2: Promediado Ponderado (TF-IDF)

$$\text{emb}_{\text{doc}} = \frac{\sum_i \text{tfidf}_i \cdot \text{emb}_i}{\sum_i \text{tfidf}_i}$$

Palabras comunes (artículos) contribuyen menos; palabras discriminativas contribuyen más.

Pros: Mitiga el impacto del ruido

Contras: Aún pierde información de orden

Opción 3: Pooling Aprendido

Permitir que la red aprenda cómo combinar embeddings de palabra:

- **Max pooling**: Tomar el máximo por dimensión
- **Attention pooling**: Aprender pesos de importancia (veremos en clases posteriores)

Pros: Más flexible y adaptativo

Contras: Requiere más datos de entrenamiento

Descripción de la Tarea

Objetivo: Predecir si una reseña de película es positiva o negativa

Dataset: IMDb (25,000 reseñas para entrenamiento, 25,000 para prueba)

Pipeline propuesto:

1. Cargar embeddings pre-entrenados (Word2Vec)
2. Tokenizar y convertir reseñas a secuencias de índices
3. Agregar embeddings (promedio simple)
4. Alimentar a clasificador simple (capas Dense)

Expectativa: ~85-90% de exactitud (limitado por promediado)

Por Qué Importan los Embeddings

- **Compacidad**: 300 dimensiones vs. 10,000+ dimensiones one-hot
- **Significado**: Capturan relaciones semánticas reales
- **Transferencia**: Úsalos en múltiples tareas
- **Eficiencia**: Pre-entrenados, listos para usar

Lecciones clave

Los embeddings transforman lo simbólico en geométrico.

Pasamos de:

- Palabras como etiquetas sin relación → **Palabras como puntos en un espacio semántico**
- Diccionarios discretos → **Espacios continuos con estructura**
- Vocabulario cerrado → **Capacidad de generalización a palabras nuevas** (especialmente FastText)

Este puente entre lenguaje y geometría es lo que permite a las redes neuronales razonar sobre significado.

Preguntas y Discusión

Gracias por la atención.

Preguntas frecuentes:

- ¿Puedo combinar múltiples esquemas de embeddings?
- ¿Qué pasa si mi idioma no tiene embeddings pre-entrenados?
- ¿Cómo detecto y mitigo sesgos en embeddings?

Limitaciones Actuales

El promediado de embeddings funciona, pero ignora:

- **Orden de palabras**: “no bueno” vs. “bueno no” → mismo embedding
- **Contexto variable**: Mismo verbo con significados diferentes según contexto
- **Dependencias a largo plazo**: Relaciones entre palabras lejanas

Papers Clásicos

- **Mikolov et al. (2013)**. “Efficient Estimation of Word Representations in Vector Space”. ICLR. [Word2Vec original]
- **Pennington, Socher & Manning (2014)**. “GloVe: Global Vectors for Word Representation”. EMNLP. [GloVe]
- **Firth, J. R. (1957)**. “A Synopsis of Linguistic Theory”. Studies in Linguistic Analysis. [Hipótesis distribucional]

Recursos Implementación

- **Gensim**: https://radimrehurek.com/gensim/
- **Spacy**: https://spacy.io/
- **TensorFlow Embedding Layer**: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Embedding

Visualización y Análisis

- **TensorBoard**: Visualización interactiva de embeddings (projector.tensorflow.org)
- **t-SNE**: van der Maaten & Hinton (2008), “Visualizing Data using t-SNE”